GitHub Actions Workflows & Bamboo Scripting Expertise Have administrative knowledge, additionally helpful. DevOps Tools like Harness, SonarQube, Nexus/Jfrog, Atlassian Tools. Knowledge on monitoring Tools like Splunk/ Thousand Eye. Understanding the Software Development Life Cycle and Agile methodologies. Strong Scripting & Coding. Good exposure with Incident Management. Production Supporting 24*7 shift rotation.

I need interview questions and answers

Here are some interview questions and answers based on your expertise in **GitHub Actions, Bamboo, DevOps tools (Harness, SonarQube, Nexus/Jfrog, Atlassian tools), monitoring tools (Splunk, ThousandEye), YAML, SaltStack, scripting, incident management, and production support**.

GitHub Actions & Bamboo

1. What are GitHub Actions, and how do they work?

GitHub Actions is a CI/CD automation tool that allows you to build, test, and deploy code within GitHub repositories. It uses YAML workflows and supports event-driven automation triggered by push, pull requests, and schedules.

- Key components:
  - **Workflows**: Define automation logic.
  - **Events**: Trigger workflows (e.g., push, PR).
  - **Jobs**: Run steps in parallel/sequence.
  - **Steps**: Execute scripts or actions.
  - **Runners**: Execute workflows (GitHub-hosted/self-hosted).

2. How do you secure GitHub Actions workflows?

- Use **secrets** (`secret.GITHUB_TOKEN`) instead of hardcoding credentials.
- Restrict repository access using **fine-grained permissions**.
- Apply **branch protection rules** to prevent unauthorized code merges.
- Use **OIDC (OpenID Connect) authentication** for cloud deployment.
- Validate inputs to **prevent command injection** in scripts.

3. How does Bamboo handle CI/CD?

Bamboo is a CI/CD tool by Atlassian that automates builds, tests, and deployments. It integrates with Jira and Bitbucket.

- **Key Features**:
  - **Plans**: Define build processes.
  - **Jobs**: Contain multiple tasks.
  - **Stages**: Organize job execution.
  - **Triggers**: Automate execution.
  - **Artifacts**: Store build outputs.

4. Difference between GitHub Actions and Bamboo?

Feature	GitHub Actions	Bamboo
Hosting	Cloud-based	On-premise/Cloud
Scripting	YAML-based	UI & script-based
Scalability	Auto-scaling runners	Requires agent setup
Integration	GitHub-centric	Supports multiple VCS
Flexibility	Native & third-party actions	Custom scripting

DevOps Tools (Harness, SonarQube, Nexus/Jfrog, Atlassian)

5. What is Harness, and how does it simplify deployments?

Harness is a CD platform that automates software releases with features like **GitOps, feature flags, progressive delivery**, and **AI-based rollback**.

- **Key Features**:
  - **Pipeline as Code** (YAML-based).
  - **Canary & Blue-Green deployments**.
  - **Auto-verification** using AI.
  - **Role-based access control (RBAC)**.

6. What is SonarQube, and why is it used in DevOps?

SonarQube is a **code quality and security analysis tool** used in CI/CD pipelines to detect bugs, vulnerabilities, and code smells.

- **Key Features**:
  - Supports **static code analysis**.
  - Measures **code coverage**.
  - Detects **security vulnerabilities** (SAST).
  - Integrates with **GitHub Actions, Jenkins, Bamboo**.

7. How do you manage artifacts using Nexus/Jfrog Artifactory?

Nexus and Jfrog Artifactory are artifact repositories used for storing libraries, Docker images, and dependencies.

- **Best Practices**:
  - Use **RBAC** for access control.
  - Configure **proxy repositories** to cache dependencies.
  - Enable **repository cleanup policies** to remove stale artifacts.
  - Automate deployment using **REST APIs**.

8. How does Jira integrate with CI/CD pipelines?

- Automate ticket updates based on build status.
- Use Jira **automation rules** for workflow transitions.
- Link commits, branches, and PRs to Jira issues.
- Generate **release notes** automatically.

Monitoring Tools (Splunk, ThousandEyes)

9. How does Splunk help in incident monitoring?

Splunk collects, indexes, and analyzes machine data for real-time insights.

- **Key Features**:
  - **Log aggregation & search** (`index=* source=*var/log/syslog`).
  - **Alerting** for error patterns.
  - **Dashboard visualizations** for metrics.
  - **Integration** with monitoring tools (e.g., Prometheus, Datadog).

10. How does ThousandEyes enhance network monitoring?

ThousandEyes is a **cloud-based network intelligence** tool that provides **synthetic monitoring, BGP route analytics, and endpoint monitoring**.

- **Use Cases**:
  - Detect **ISP outages & packet loss**.
  - Monitor **DNS resolution & latency**.
  - Analyze **SaaS & cloud application performance**.

Configuration Management (YAML, SaltStack)

11. What is YAML, and why is it used in DevOps?

YAML (Yet Another Markup Language) is **human-readable data format** used in CI/CD workflows, Kubernetes manifests, and infrastructure as code (IaC).

- **Example YAML Workflow (GitHub Actions)**:

```yaml
name: CI Pipeline
on: [push, pull_request]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v2
      - name: Run tests
        run: pytest tests/
```

12. What is SaltStack, and how does it work?

SaltStack is an **automation and configuration management** tool that uses a master-minion architecture.

- **Key Features**:
  - Uses **Salt States** for declarative automation.
  - Supports **event-driven automation**.
  - Manages **infrastructure as code (IaC)**.
- **Example Salt State**:

```yaml
install_nginx:
  pkg.installed:
    - name: nginx
```

Scripting, Incident Management, & Production Support

13. What scripting languages have you used in DevOps?

- **Bash**: Automating server tasks (e.g., cron jobs, log rotation).
- **Python**: Writing infrastructure automation scripts.
- **PowerShell**: Managing Windows-based infrastructure.
- **Groovy**: Used in Jenkins pipelines.

14. How do you handle an incident in a production system?

- **Detect**: Use monitoring tools (Splunk, Prometheus).
- **Diagnose**: Check logs, alerts, and system metrics.
- **Escalate**: Notify relevant teams if needed.
- **Resolve**: Apply fixes, rollback if required.
- **Postmortem**: Conduct RCA (Root Cause Analysis) and document lessons learned.

15. How do you handle deployments in a 24x7 production support model?

- Schedule **low-risk deployments** during maintenance windows.
- Use **blue-green/canary deployments** to minimize downtime.
- Implement **feature flags** for safer rollouts.
- Maintain **runbooks** for troubleshooting.

16. How do you ensure high availability in prod*7 Uses?

- Deploying in **multiple availability zones**.
- Implement **auto-scaling** for peak loads.
- Use **load balancers** for failover.
- Set up **disaster recovery (DR) strategies**.

Would you like me to add any specific scenario-based questions?

yes

Here are some **scenario-based interview questions**! Along with detailed answers. These questions will test your practical experience in **GitHub Actions, Bamboo, DevOps tools, monitoring, scripting, incident management, and production support**.

GitHub Actions & Bamboo Scenarios

1. Scenario: A GitHub Actions workflow suddenly stops executing, but there are no errors in the logs. How would you debug this?

Solution:

- Check if the workflow was **disabled manually** in GitHub → Actions tab → Workflow settings.
- Verify **repository settings** to ensure actions are not restricted by organizational settings.
- Check **GitHub Actions Runner status** (for self-hosted runners) using:

```
bash
./run.sh status
```

- Review the **workflow execution history** for skipped jobs due to `if` conditions.
- Check **GitHub API rate limits** (`curl -H "Authorization: token <token>" https://api.github.com/rate_limit`).
- Re-run workflow manually to see if it triggers correctly.

---

2. Scenario: A Bamboo build fails intermittently without code changes. How would you troubleshoot?

Solution:

- Check Bamboo Agent logs (`atlassian-bamboo-agent.log`) for transient issues.
- Increase **task execution timeout** in the build settings.
- Analyze **external dependencies** (e.g., network failures affecting package downloads).
- Investigate **concurrent builds** that may be affecting shared resources.
- Run tests locally to rule out environment-related issues.

DevOps Tools Scenarios (Harness, SonarQube, Nexus/Jfrog, Atlassian)

3. Scenario: A SonarQube scan takes too long to complete in a CI/CD pipeline. How would you optimize it?

Solution:

- Exclude unnecessary directories from scanning (`sonar.exclusions=**/*test**/**/node_modules/**`).
- Use **SonarQube caching** to avoid re-analyzing unchanged files.
- Increase **worker threads** using `sonar.threadcount`.
- Run **incremental analysis** (`sonar.analysis.mode=incremental`) instead of full scans.
- Ensure **SonarQube server is properly sized** for large projects.

---

4. Scenario: A Nexus/Jfrog Artifactory repository is running out of storage. What actions would you take?

Solution:

- Enable **artifact cleanup policies** to delete old or unused artifacts.
- Use **Nexus Pro's "Blob Store Cleanup" feature** to remove orphaned files.
- Configure **repository retention rules** (e.g., keep only the last 5 builds).
- Migrate to an **external storage solution** like Amazon S3.
- Optimize **storage with deduplication** for Docker images.

Monitoring (Splunk, ThousandEyes) Scenarios

5. Scenario: Your application latency spikes, and you need to investigate using Splunk. How would you proceed?

Solution:

- Search for **error patterns in logs** using:

```
bash
tail -f /var/log/app/error.log | stats count by status
```

- Check for **database slow queries** in logs.
- Analyze **infrastructure metrics** (CPU, memory) in Splunk dashboards.
- Correlate latency issues with **recent deployments**.
- Set up **real-time alerts** for high latency transactions.

---

6. Scenario: A user reports slow connectivity to a cloud application. How would you use ThousandEyes to diagnose the issue?

Solution:

- Run a **network path visualization** to check for high latency hops.
- Analyze **packet loss & jitter** between the user's ISP and cloud provider.
- Check **BGP route changes** that may be affecting routing.
- Run a **synthetic test** to simulate user access from different locations.
- Validate **DNS resolution speed** for potential domain-related issues.

Configuration Management & Scripting (YAML, SaltStack)

7. Scenario: A SaltStack state fails with an error message: "No matching sls found for 'install_nginx'." How do you troubleshoot?

Solution:

- Ensure the SLS file exists (`/srv/salt/install_nginx.sls`).
- Verify **minion connectivity** to the master (`salt '*' test.ping`).
- Check for **typos** in the state file.
- Manually apply the state and check for detailed errors:

```
bash
salt '*' state.apply install_nginx test=True
```

- Restart Salt services (`systemctl restart salt-master`).

---

8. Scenario: A YAML-based Kubernetes deployment fails due to indentation errors. How do you fix it?

Solution:

- Validate YAML syntax using:

```
bash
yamllint deployment.yaml
```

- Check for **mixing tabs vs. spaces** (YAML is space-sensitive).
- Use an online YAML validator (e.g., YAML Lint).
- Ensure proper key-value nesting in YAML.

Incident Management & Production Support Scenarios

9. Scenario: A critical production server crashes at midnight. How would you respond?

Solution:

- Acknowledge the incident & notify stakeholders.
- Check system health using monitoring tools (Splunk, Prometheus).
- Gather logs & error messages from the crashed server.
- If non-critical, rollback recent deployments.
- Apply a temporary fix and schedule a postmortem.

---

10. Scenario: A high-priority production issue is escalated, and you need to coordinate with multiple teams. How do you handle it?

Solution:

- Join the incident bridge call immediately.
- Assign roles (Incident Commander, Communications, Troubleshooters).
- Follow escalation procedures (e.g., engaging DB, network teams).
- Document all actions taken in a live incident report.
- Communicate resolution updates to stakeholders in real-time.

---

11. Scenario: A deployment causes downtime, and you need to rollback. How do you proceed?

Solution:

- Assess the impact and confirm rollback feasibility.
- Trigger rollback process (e.g., redeploy previous stable build).
- Verify rollback success (monitor logs, metrics, alerts).
- Communicate with stakeholders (developers, business teams).
- Perform RCA to prevent future occurrences.

Final Thoughts

Would you like me to focus on specific areas like **Terraform, Kubernetes, Azure DevOps**, or any other? Let me know! Future interviews scenario-based questions?

ChatGPT can make mistakes. Check important info.